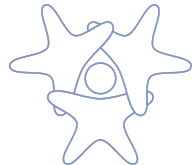


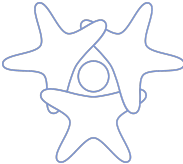


Disciplina: Engenharia de Software e Gestão de Times Ágeis

Fundamentos e Conceitos da
Engenharia de Software

Prof. Pedro Rodolfo Gomes de Souza
prgsouza3@gmail.com



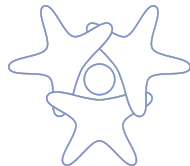


UNINASSAU
Campus Boa Viagem

Evolução de Software

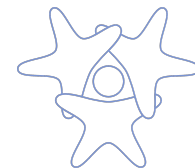
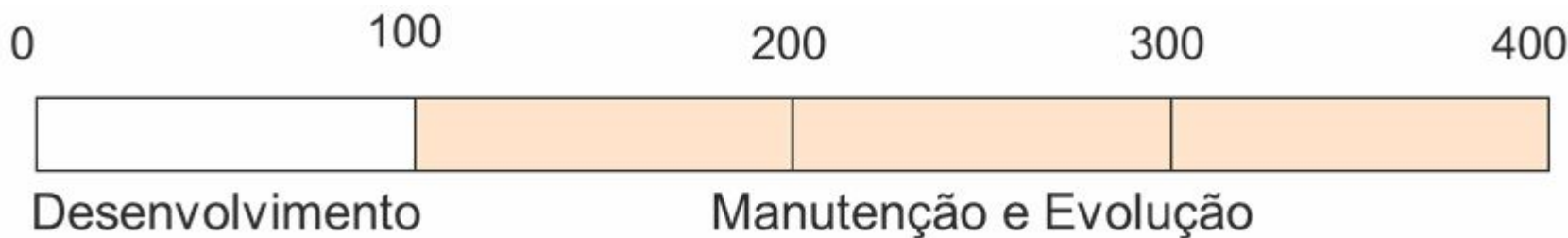
Evolução de Software

- Sistemas de software são caros e geralmente têm vida útil longa
 - Empresas precisam usá-los por anos para ter retorno do investimento
- Depois de implantados, sistemas devem inevitavelmente mudar para permanecerem úteis



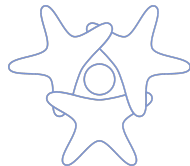
Custo de Evolução

- O custo de manutenção é muito maior que o custo de desenvolvimento
 - 60% a 90% dos custos do software são relativos à manutenção
 - 75% dos profissionais de software estão envolvidos com atividades de evolução

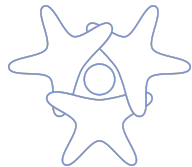
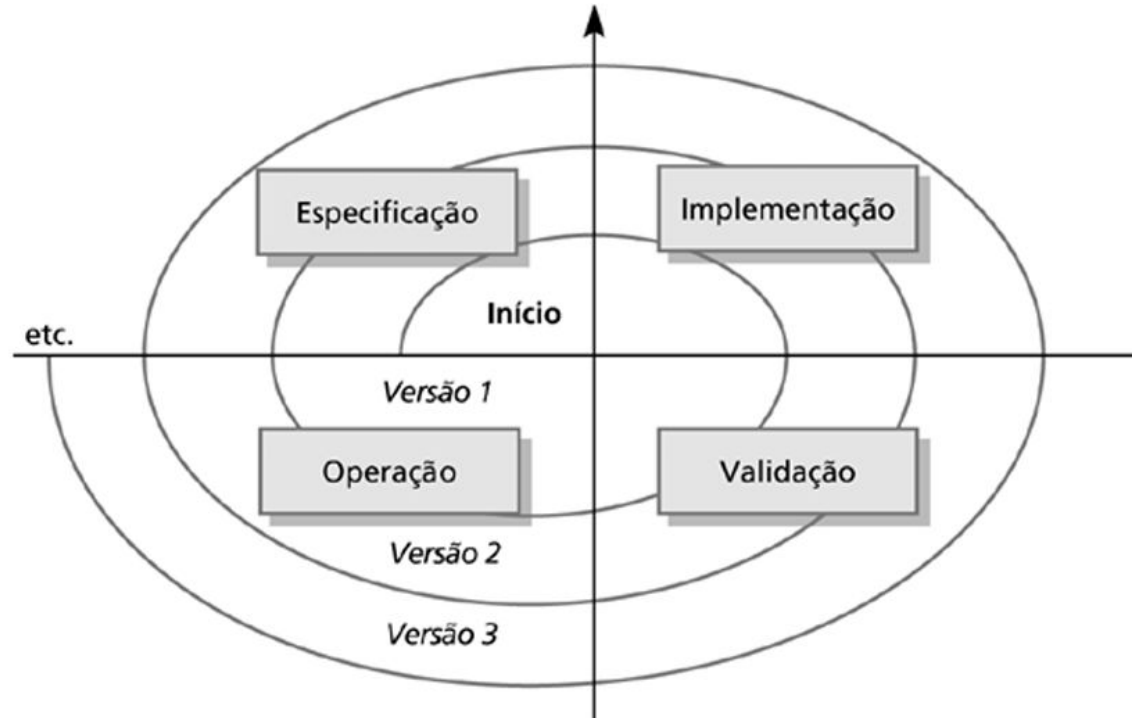


Mudança Contínua

- Mudanças no ambiente requerem atualizações do sistema
 - Ao implantar um sistema modificado, este sistema causa uma mudança no ambiente
- Ciclos contínuos e ininterruptos de desenvolvimento e evolução podem ser representados por uma espiral

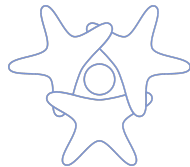


Modelo Espiral de Evolução



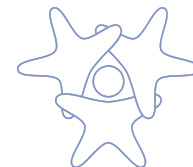
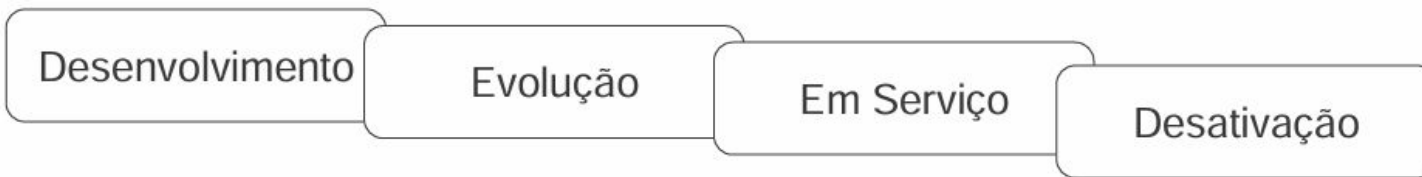
Fases do Sistema

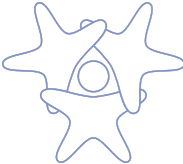
- Desenvolvimento de Software
 - Primeira solicitação do cliente
- Evolução
 - Sistema é intensamente usado
- Em Serviço
 - Sistema é pouco usado
- Desativação
 - Empresa considera a substituição do sistema



Evolução vs. Em Serviço

- Evolução
 - Mudanças significativas são feitas tanto na arquitetura quanto nas funcionalidades
 - A estrutura tende a gradativamente se degradar
- Em Serviço
 - Apenas mudanças pequenas e essenciais ocorrem (software é pouco usado)



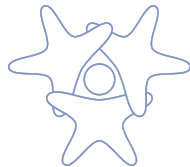


UNINASSAU
Campus Boa Viagem

Processos de Evolução

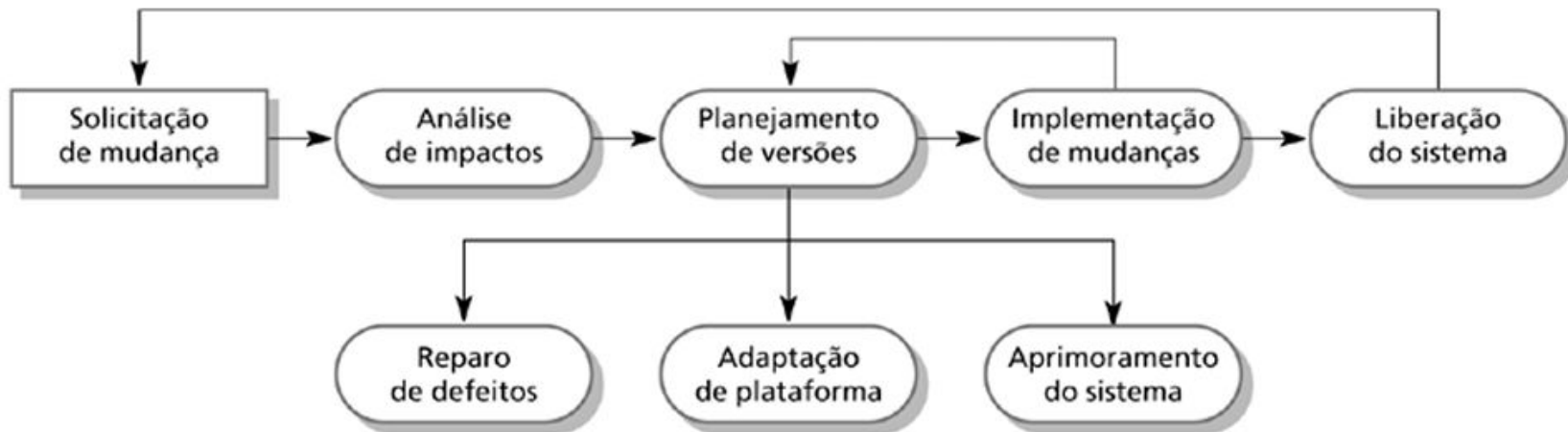
Processo de Evolução

- A evolução pode variar entre organizações e tipos de sistemas
 - Processo informal: solicitação de mudanças provém de conversas entre usuários e desenvolvedores
 - Processo formalizado: documentação produzida e avaliada em cada atividade do processo



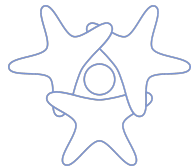
Modelo de Processo

- Cada solicitação de mudança é avaliada, planejada e implementada
 - O processo se repete em novas solicitações



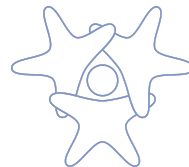
Análise e Planejamento

- Análise de Impactos
 - Avalia o custo e a porção do sistema afetado pela mudança solicitada
 - Decide se a mudança será aceita
- Planejamento de Versões
 - Caso sejam aceitas, diferentes tipos de mudanças podem ser agrupados para a próxima versão
 - Correção de defeitos, adaptações de plataforma e aperfeiçoamentos



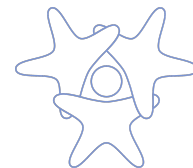
Análise e Planejamento

- Análise de Impactos
 - Avalia o custo e a porção do sistema afetado pela mudança solicitada
 - Decide se a mudança será aceita
- Planejamento de Versões
 - Caso sejam aceitas, diferentes tipos de mudanças podem ser agrupados para a próxima versão
 - Correção de defeitos, adaptações de plataforma e aperfeiçoamentos



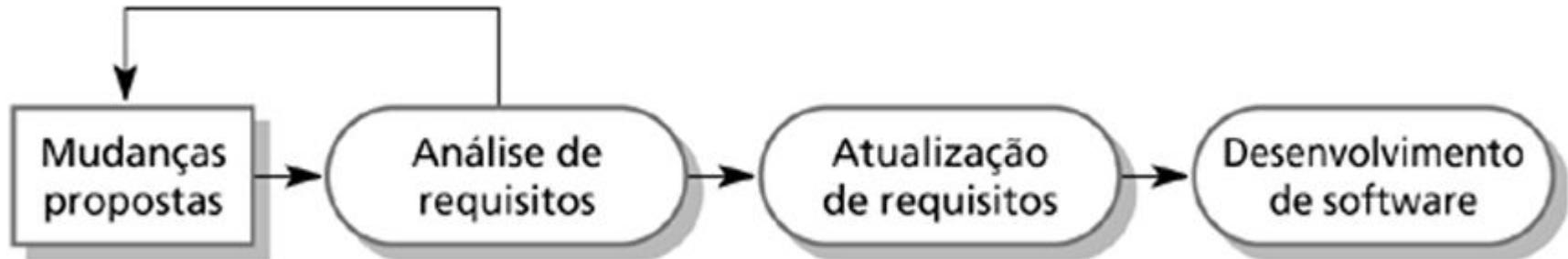
Implementação e Liberação

- Implementação de Mudanças
 - Geralmente, uma nova versão inclui um conjunto com várias mudanças solicitadas
- Liberação do Sistema
 - Uma nova versão é liberada após validação com o cliente
- O processo se repete com um novo conjunto de solicitação de mudanças



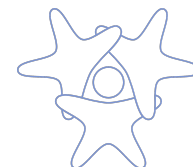
Implementação de Mudanças

- Lembrando que a implementação de mudanças deve atualizar a documentação correspondente
 - Especificação, modelos de projeto, implementação, testes, etc



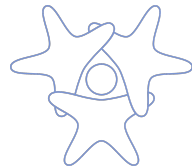
Compreensão de Programa

- A implementação da mudança envolve atividades semelhante ao desenvolvimento inicial do sistema
 - Uma diferença fundamental é ser necessário compreender o programa
- Compreensão inclui
 - Entender como a funcionalidade a ser alterada encontra-se implementada
 - Avaliar o impacto da mudança em outras partes do programa



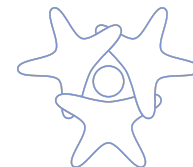
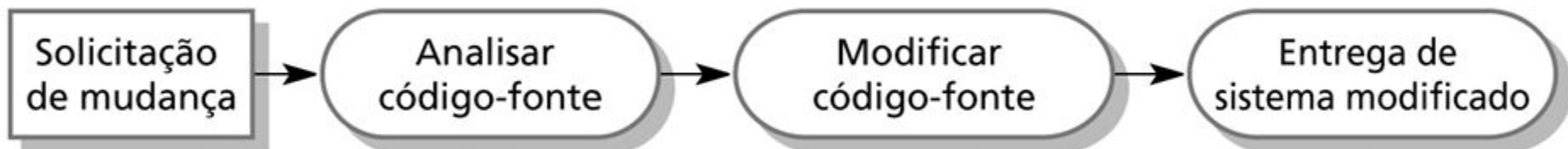
Situações Emergenciais

- Algumas solicitações de mudanças podem ser urgente, exemplo:
 - Se ocorrer um defeito grave que impede o funcionamento normal do sistema
 - Alterações do ambiente impedem o funcionamento do sistema
 - Mudanças no negócio do cliente ou uma nova legislação
 - Entrada de um novo concorrente no mercado



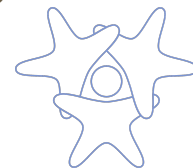
Reparo Emergencial

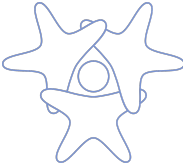
- Pode não haver tempo hábil para atualização da documentação
 - Ao invés de seguir as atividades “normais” de desenvolvimento, a alteração é feita diretamente no código



Refazer a Correção

- Código para correção de emergência geralmente tem baixa qualidade
- Portanto, a correção de emergência deveria ser idealmente re-implementada
 - E a documentação correspondente atualizada
- Na prática, este re-trabalho tem baixa prioridade e acaba esquecido



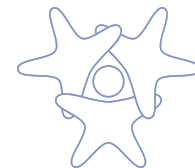


UNINASSAU
Campus Boa Viagem

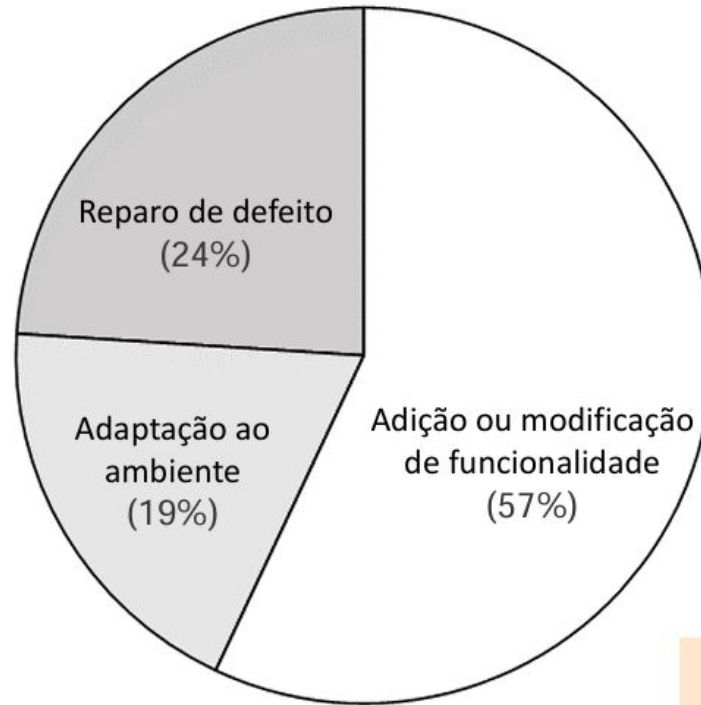
Manutenção de Software

Manutenção de Software

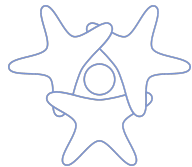
- Processo geral de modificar o sistema após a sua entrega
- Três tipos principais de manutenção
 - Manutenção corretiva: mudanças para reparo de defeitos de software
 - Manutenção adaptativa: mudanças para adaptar o software a outro ambiente
 - Manutenção perfectiva: mudanças para adicionar funcionalidade ao sistema



Esforço de Manutenção

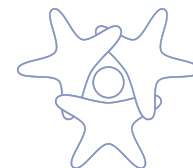


Estudo de Davidsen
e Krogstie (2010)



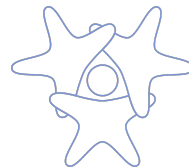
Desenvolvimento e Manutenção

- O desenvolvimento e a manutenção devem ter a mesma importância
- É economicamente compensador investir esforço no desenvolvimento para reduzir custos de manutenção
 - É mais caro adicionar a funcionalidade depois que o sistema entra em operação



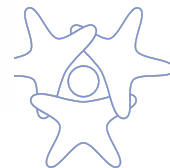
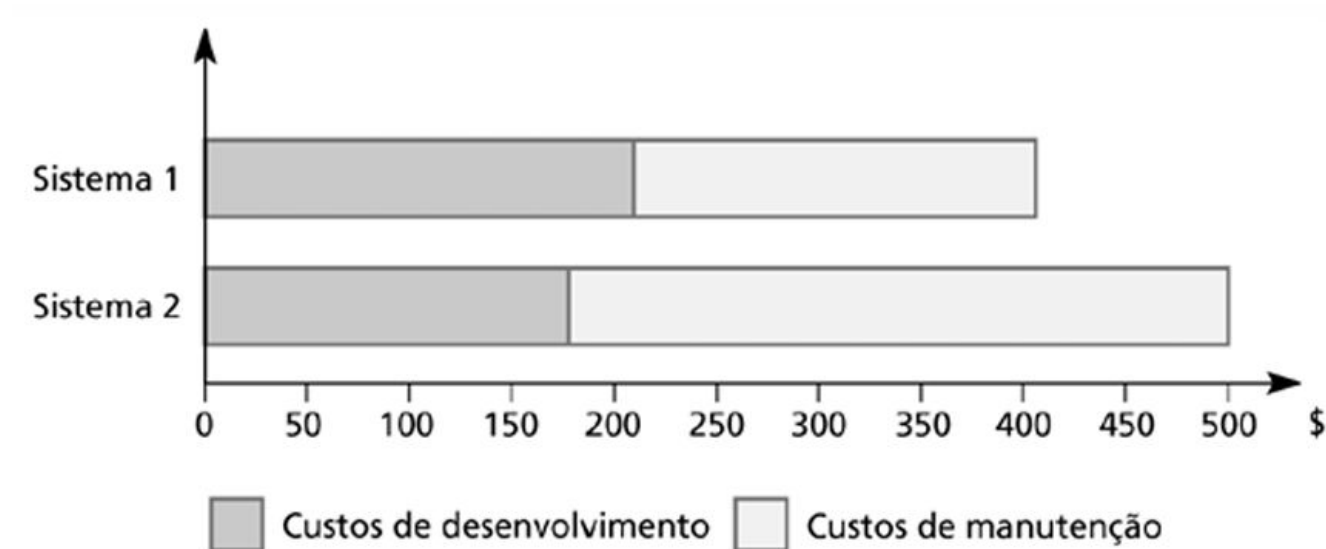
Adição de Funcionalidade

- Razões para o alto custo de acrescentar a funcionalidade durante a manutenção
 - Um novo time precisa entender o programa
 - O software pode não ter sido desenvolvido para fácil manutenção
 - Tarefas de manutenção não são atrativas
 - A medida que o sistema envelhece, a sua estrutura se degrada



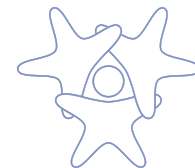
Retorno do Investimento

- Sistema 1 têm investimento maior na fase de desenvolvimento



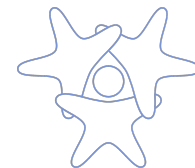
Por que não se investe?

- Muitas empresas relutam em gastar mais no desenvolvimento
 - Podendo reduzir o custo de manutenção
- Razões principais
 - Empresas fazem planos de investimento de curto prazo
 - Os desenvolvedores geralmente não são responsáveis por manter o sistema



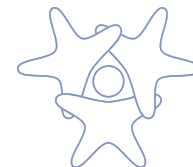
Investir no Desenvolvimento

- Investir em técnicas de Engenharia de Software ajudam a reduzir o custo de manutenção
- Exemplos de investimento
 - Especificação cuidadosa do software
 - Desenvolvimento de testes automatizados
 - Uso de ferramentas e orientação a objetos
 - Gerência de configuração, etc



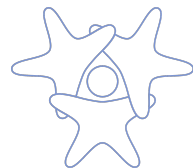
Previsão de Manutenção

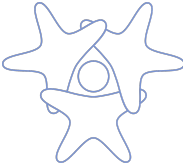
- É importante avaliar as mudanças que podem ser necessárias no sistema
- Perguntas para apoiar a previsão
 - Quais solicitações de mudança são esperadas?
 - Quais partes são mais suscetíveis a mudanças?
 - Quais partes do sistema são mais caras de serem mantidas?



Complexidade e Custo

- Quanto mais complexo é o sistema, mais cara é a sua manutenção
- Exemplos de métricas
 - Quantidade de solicitações de manutenção corretiva
 - Quantidade de mudanças pendentes
 - Tempo médio para análise de impacto
 - Tempo médio para realizar a mudança



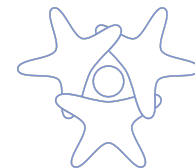


UNINASSAU
Campus Boa Viagem

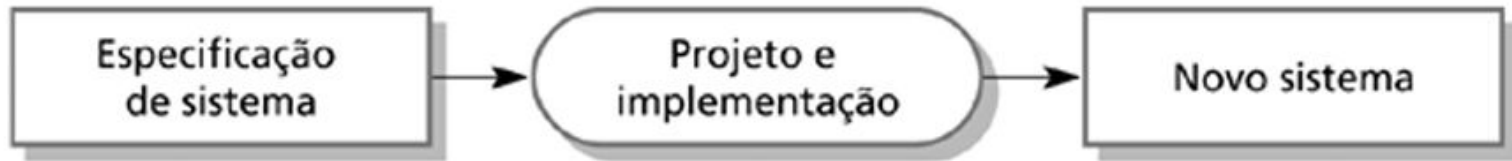
Reengenharia, Refatoração e Bad Smell

Reengenharia de Sistemas

- Reestruturação ou reescrita de parte ou de todo um sistema legado sem mudança na sua funcionalidade
 - O objetivo é melhorar a qualidade
- O sistema pode ser reestruturado e/ou redocumentado
 - Aplicável a subsistemas ou sistemas de grande porte que necessitam de manutenção frequente



Engenharia x Reengenharia

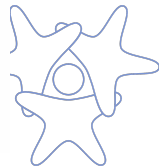


Engenharia direta



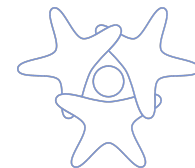
Reengenharia de software

Sommerville9

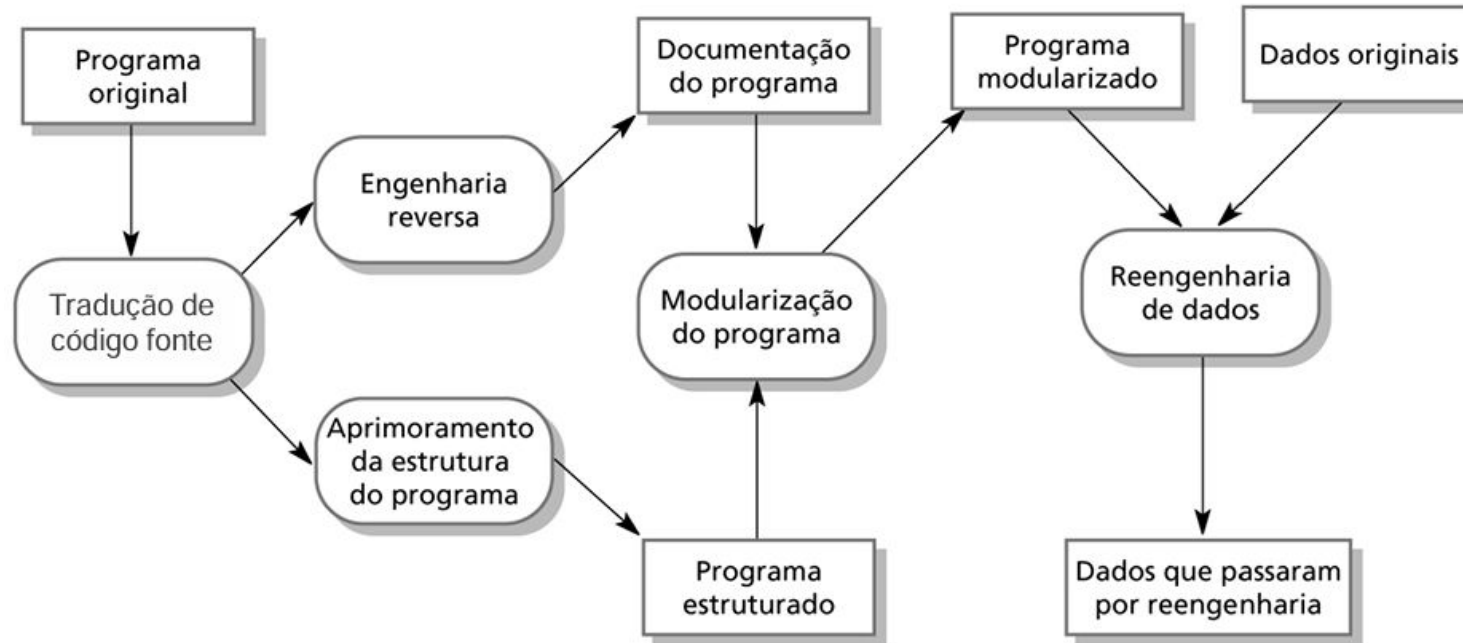


Vantagens da Reengenharia

- Menor Risco
 - O risco de desenvolver o sistema novamente é alto
 - Podem ocorrer erros de especificação ou problemas no desenvolvimento
- Menor Custo
 - O custo de reengenharia pode ser muito menor que o custo de desenvolvimento
 - Ferramentas podem automatizar parcialmente

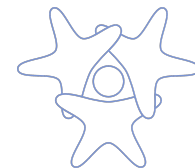


Processo de Reengenharia



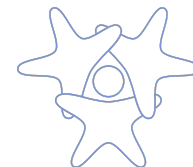
Atividades de Reengenharia (1)

- Tradução de código-fonte
 - Converter o código para uma nova linguagem
- Engenharia reversa
 - Analisar o programa para compreendê-lo e documentá-lo
- Melhoria da estrutura (refatoração)
 - Analisar e modificar a estrutura para facilidade de entendimento



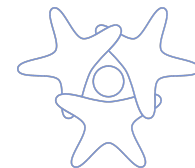
Atividades de Reengenharia (2)

- Modularização de programa
 - Reorganizar a estrutura do programa para torná-lo mais modular
 - Partes redundantes são identificadas e removidas
- Reengenharia de dados
 - Limpar e reestruturar os dados (ou estruturas de dados) do sistema
 - Conversão do banco de dados



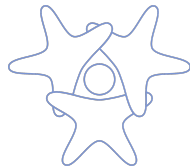
Bad Smell

- Bad smell é uma situação no qual a estrutura do programa pode ser melhorada com refatoração
- Exemplos de bad smells
 - Código Duplicado
 - Classes ou métodos longos



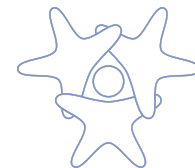
Refatoração

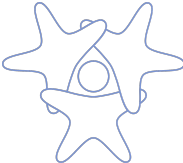
- Processo de alterar a estrutura de um programa sem mudar o seu comportamento observável
 - Objetivo é reduzir a complexidade e torná-lo mais compreensível
- Refatoração pode ser visto como uma manutenção preventiva
 - Nenhuma nova funcionalidade é incluída nem há adaptações ou correções



Refatoração e Métodos Ágeis

- Refatoração é muito praticada na Programação Extrema (XP)
 - Reflexo das mudanças contínuas que podem degradar a estrutura do sistema
- Alterar a estrutura do programa podem causar novos erros
 - O uso frequente de testes ajudam na detecção de erros de refatoração



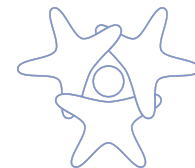


UNINASSAU
Campus Boa Viagem

Requisitos de Usuário e Requisitos de Sistema

O que são requisitos?

- Requisitos
 - Descrições dos serviços prestados pelo sistema e das restrições a sua operação
- Engenharia de Requisitos
 - Processo de descobrir, analisar, documentar e verificar requisitos



Visões da Especificação

- É interessante gerar diferentes visões da especificação de sistema
 - Comunicação com diferentes tipos de interessados (stakeholders)



Requisitos do Usuário



Gerentes do cliente
Usuários finais
Engenheiros do cliente
Fornecedores

Requisitos de Sistema

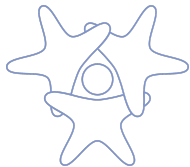


Analistas do sistema
Arquitetos de software
Desenvolvedores



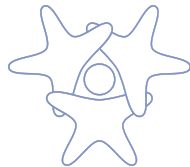
Requisitos de Usuário

- Descreve as funções e restrições do sistema de forma abstrata
 - Inteligível pelo usuário / cliente
- Ponto de vista das necessidades da empresa cliente
 - Não indica uma solução
- Escrito em linguagem natural com diagramas simples (ex. tabelas)



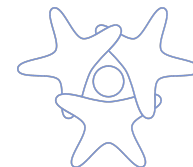
Requisitos de Sistema

- São descrições mais detalhadas que os requisitos do usuário
- Devem ser padronizados, completos e consistentes
 - Usados pela equipe de desenvolvimento
 - Pode fazer parte do contrato



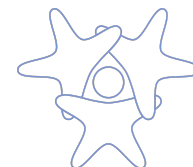
Linguagem Natural

- Não é fácil padronizar os requisitos usando linguagem natural
- Na escada rolante dizia
 - “Calçados devem ser usados”
 - “Cachorros devem ser carregados”



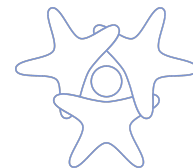
Alguns Problemas

- Falta de clareza
 - Pode ser difícil uma linguagem precisa e não ambígua
- Confusão de requisitos
 - Requisitos pode estar misturados com informações do projeto
- Fusão de requisitos
 - Diversos requisitos expressos juntos



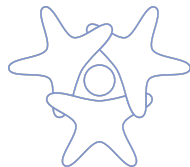
Diretrizes Gerais de Redação

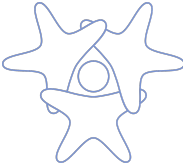
- Adotar um formato padrão e usá-lo em todas as definições de requisitos
- Usar a linguagem de forma simples e consistente (pode x deve)
- Usar realce (negrito, itálico ou sublinhado) para partes importantes
- Evitar usar linguagem técnica de informática em requisitos de usuário



Especificação Estruturada

- É uma forma restrita da linguagem natural
- Vantagens
 - Mantém a facilidade de expressão e compreensão da linguagem natural
 - Garante algum grau de uniformidade na especificação
 - Pode ser escrita em formulários especiais



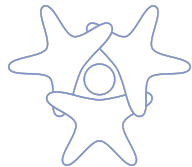


UNINASSAU
Campus Boa Viagem

Requisitos Funcionais e Requisitos Não Funcionais

Requisitos Funcionais

- Declarações de serviços que o sistema deve fornecer
- Documenta
 - como o sistema deve reagir a entradas específicas
 - como deve se comportar em determinadas situações
 - o que o sistema não deve fazer



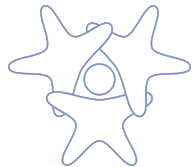
Atributos dos Requisitos Funcionais

- Completude
 - Todos os serviços devem estar definidos
- Coerência
 - Os requisitos não devem ter definições contraditórias
- Na prática, é quase impossível atingir completude e coerência dos requisitos



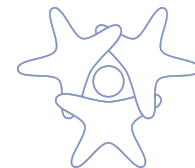
Exemplos de Requisitos Funcionais

- O usuário pode fazer uma busca na lista de consultas de todas as clínicas
- O sistema deve gerar, a cada dia e para cada clínica, uma lista de pacientes
- Cada membro da equipe que utiliza o sistema deve ser identificado por um número de funcionário de oito dígitos



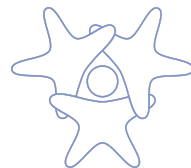
Ambiguidades em Requisitos

- A imprecisão na especificação de requisitos é motivo de vários problemas
 - O desenvolvedor tende a interpretar o requisito da maneira mais fácil de implementar
- “O usuário pode fazer uma busca na lista de consultas...”
 - Quais as entradas da busca?
 - O que será retornado?



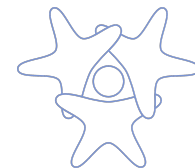
Requisitos Não-Funcionais (RNF)

- Restrições sobre os serviços ou funções do sistema
 - Exemplos: confiabilidade, desempenho, espaço em disco
- Podem ser do sistema todo ou de partes do sistema
- Requisitos não-funcionais podem ser mais críticos que requisitos funcionais
 - Se não satisfaz, o sistema é inútil



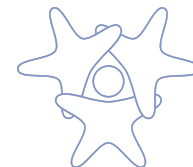
Classificação de RNF

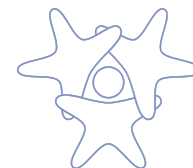
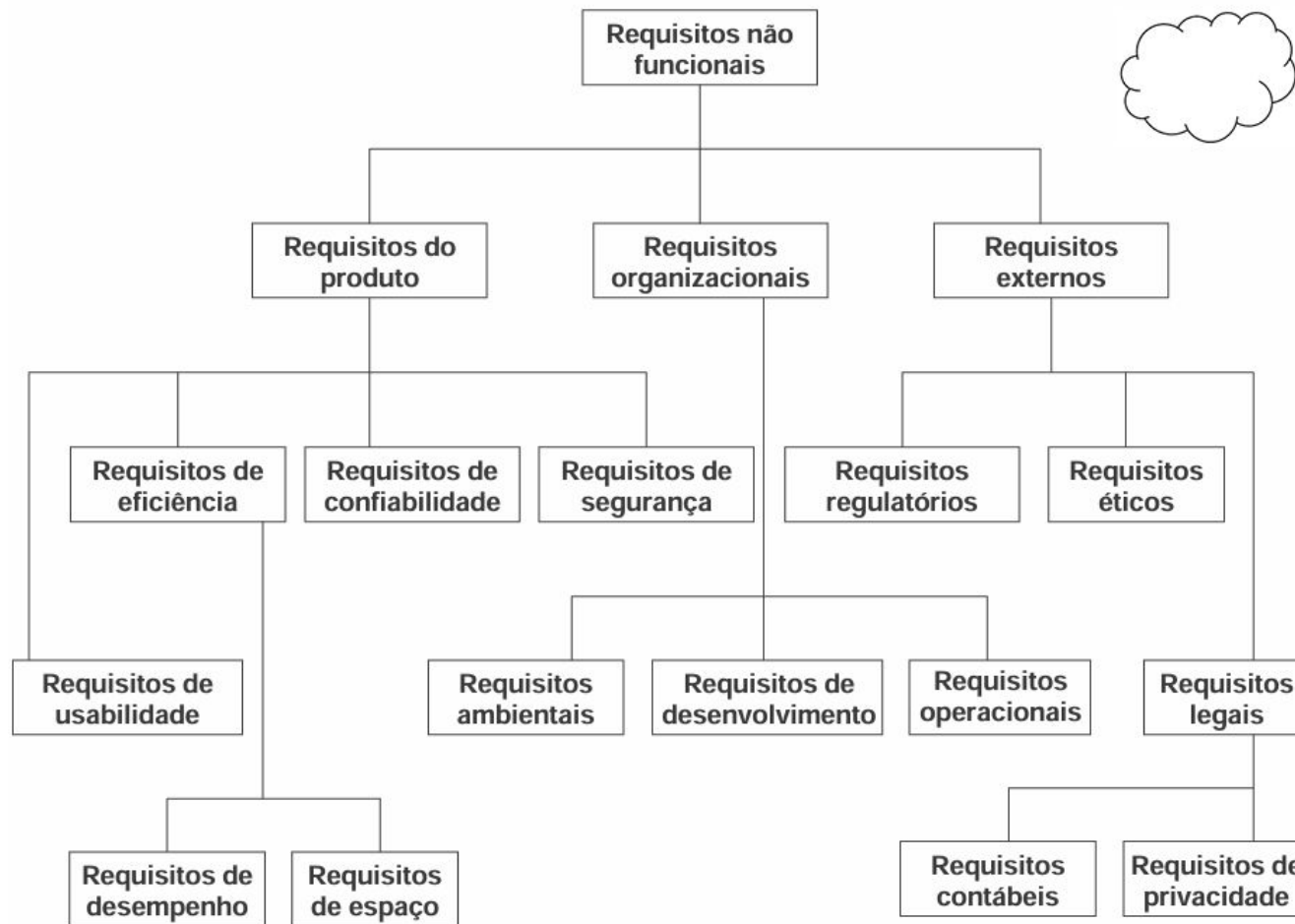
- Requisitos do Produto
 - Especificam o comportamento do software (ex.: desempenho)
- Requisitos Organizacionais
 - Consequência de políticas e procedimentos das empresas (ex.: padrões do cliente)
- Requisitos Externos
 - Derivados do ambiente ou fatores externos ao sistema (ex.: legislação)



Exemplos de RNF

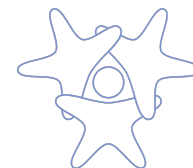
- Requisitos do Produto
 - Rapidez que o sistema deve executar uma solicitação (tempo de resposta)
- Requisitos Organizacionais
 - Cada membro é identificado por um número de funcionário de oito dígitos
- Requisitos Externos
 - Informações pessoais dos usuários não podem ser vistas pelos operadores do sistema (privacidade)





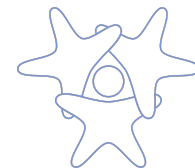
Verificação de RNF

- Às vezes são de difícil verificação
- Idealmente, requisitos não-funcionais devem ser mensuráveis
 - Após a implementação, estes podem ser testados objetivamente



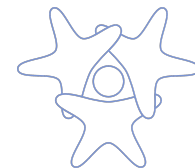
Métricas de RNF

- Velocidade
 - Transações processadas por segundo
 - Tempo de resposta
 - Tempo de atualização de tela
- Facilidade de uso
 - Tempo de treinamento
 - Número de quadros de ajuda



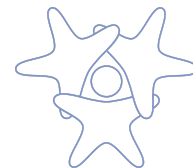
Métricas de RNF

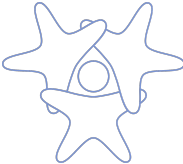
- Confiabilidade
 - Tempo médio para falhar
 - Probabilidade de indisponibilidade
 - Taxa de ocorrência de falhas
 - Disponibilidade
- Robustez
 - Tempo para reiniciar após uma falha
 - Porcentagem de eventos que causam falhas



Alguns Problemas de RNF

- A especificação quantitativa de requisitos não funcionais é difícil
- Ocorre mistura de requisitos funcionais e não funcionais
- Requisitos não funcionais podem conflitar com outros requisitos (funcionais ou não)



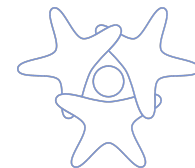


UNINASSAU
Campus Boa Viagem

Engenharia de Requisitos

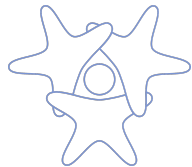
Stakeholders

- Pessoas que têm qualquer influência direta ou indireta sobre os requisitos
 - Clientes e usuários
 - Gerentes de projeto
 - Analistas de sistema
 - Engenheiros de testes
 - Mantenedores, etc.

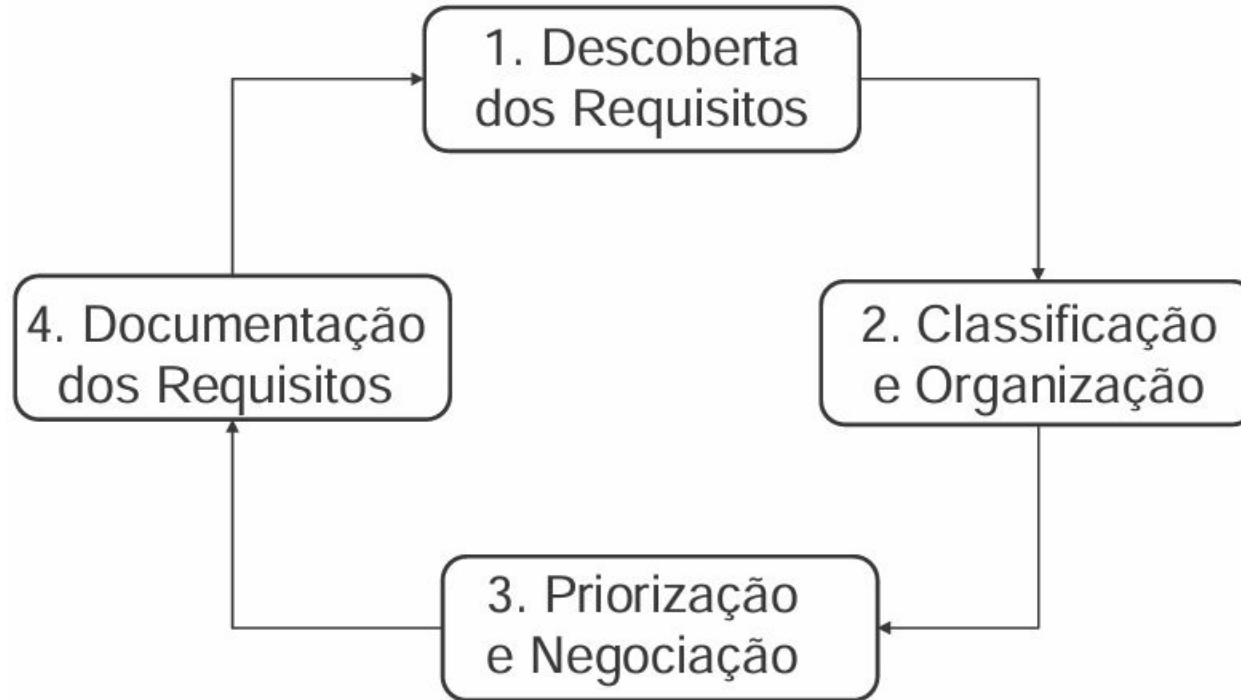


Eng. de Requisitos: Atividades

- Inclui três atividades principais
 - Elicitação (e análise) de requisitos
 - Especificação de requisitos
 - Validação dos requisitos
- Estas atividades seguem um processo iterativo, com atividades intercaladas

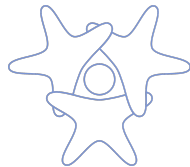


Elicitação e Análise de Requisitos



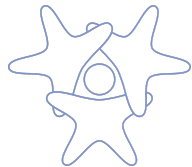
Descoberta de Requisitos

- O objetivo é descobrir
 - O domínio de aplicação
 - Serviços que devem ser fornecidos pelo sistema
 - Restrições associadas ao domínio ou aos serviços
- Várias técnicas podem ser usadas
 - Envolvem diversos stakeholders



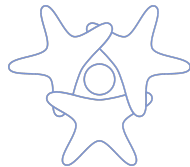
Especificação de Requisitos

- Processo de escrever os requisitos de usuário e do sistema em um documento de requisitos
- Notações para escrever requisitos
 - Linguagem natural
 - Linguagem natural estruturada
 - Notações gráficas (exemplo, UML)
 - Especificação matemática



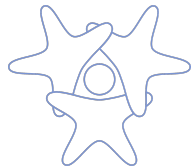
Seções Principais do Documento

1. Prefácio e Glossário
2. Introdução
3. Requisitos de usuários
4. Arquitetura do sistema
5. Requisitos do sistema
6. Modelos do sistema
7. Itens adicionais (Evolução, Apêndices, Índice, etc.)



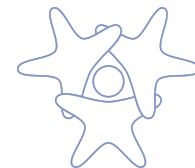
Validação de Requisitos

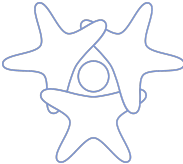
- Mostrar que os requisitos realmente definem o sistema que o cliente deseja
- Descobrir problemas com os requisitos



Verificações da Validação

- Verificações de validade
 - Quais serviços são necessários?
- Verificações de consistência
 - Existe conflitos entre requisitos?
- Verificações de completude
 - Todos os requisitos estão documentados?
- Verificações de realismo
 - Os requisitos podem ser implementados?
- Facilidade de verificação
 - Como verificar se o requisito foi implementado?



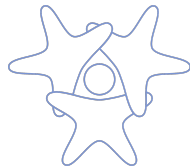


UNINASSAU
Campus Boa Viagem

Técnicas de Elicitação e Especificação de Requisitos

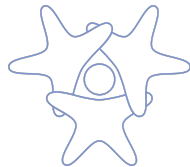
Elicitação e Especificação

- Técnicas para levantamento e documentação de requisitos
 - Entrevistas
 - Etnografia
 - Histórias e Cenários
 - Casos de Uso



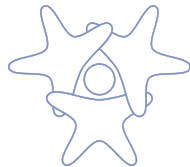
Entrevistas

- Questionar os stakeholders sobre o sistema (ou processo) atual e sobre o sistema que será desenvolvido
- Tipos de entrevistas
 - Entrevistas fechadas: conjunto pré definido de perguntas
 - Entrevistas abertas: sem agenda pré definida; se adapta para explorar o conhecimento do stakeholder



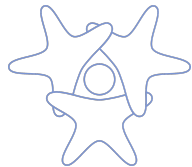
Etnografia

- É uma técnica de observação utilizada para compreender os requisitos sociais e organizacionais
- O analista (engenheiro de requisitos) se insere na organização do cliente
 - Observa o trabalho no dia a dia
 - Anota as tarefas dos funcionários



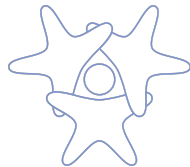
Etnografia é eficaz...

- Para descobrir como as pessoas realmente trabalham
- Para descobrir a cooperação e conscientização das atividades de outras pessoas
- Para desenvolver um protótipo
- Para descobrir importantes detalhes que outros métodos omitem



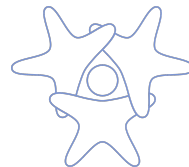
Histórias e Cenários

- Descreve uma situação de uso do sistema
- Inclui informações como
 - Nome do Cenário
 - Ator(es)
 - Pré-condição
 - Fluxo normal
 - Fluxos alternativos
 - Pós-condição



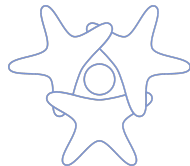
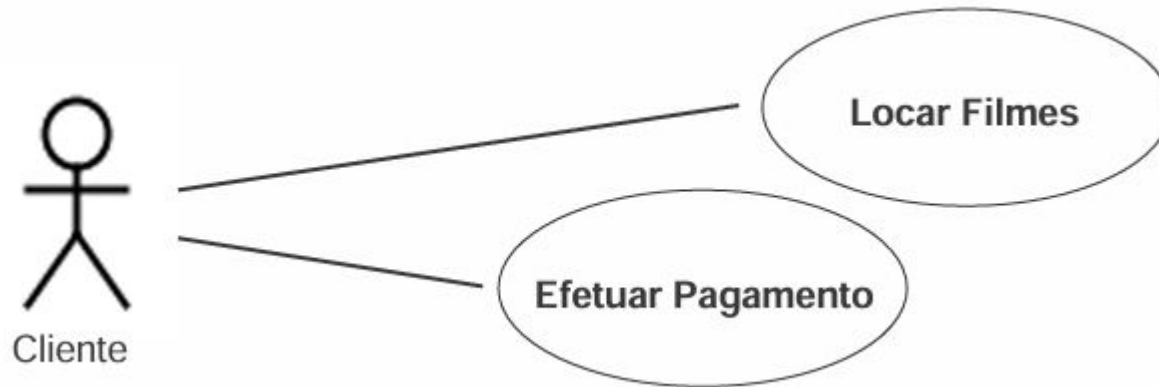
Exemplo de Cenário (ATM)

- Nome do Cenário: Sacar dinheiro
- Ator: Correntista
- Pré-condição: Conta e senha validada
- Fluxo normal
 - 1. Entrar com valor do saque
 - 2. Confirmar dados e operação
 - 3. Debitar valor da conta do cliente
- Fluxo alternativo: Saldo insuficiente
 - 3.1 Apresentar aviso ao cliente
- Pós-condição:
 - Valor sacado é debitado do saldo do cliente



Casos de Uso

- Casos de Uso identificam
 - Os atores envolvidos
 - As funcionalidades principais
 - A interação entre atores e funcionalidades do sistema





UNINASSAU
Campus Boa Viagem

Arquitetura de Software

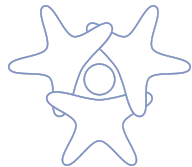
Projeto Arquitetural

- O projeto é uma atividade criativa
 - Cada arquiteto tem sua própria maneira de projetar o software
- Projeto arquitetural faz a ligação entre
 - Requisitos (domínio do problema)
 - Projeto detalhado (domínio da solução)



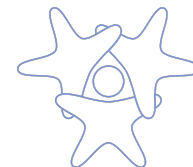
Definindo a Solução

- Primeiro passo do projeto de sistema
- É frequentemente conduzido em paralelo com atividades de especificação de requisitos
- O projeto arquitetural envolve
 - Identificação dos componentes principais do sistema
 - Definição das interfaces de comunicação entre os componentes



Vantagens da Arquitetura

- Comunicação entre stakeholders
 - A arquitetura pode ser usada como foco da discussão sobre o sistema
- Análise de sistema
 - Adequação aos requisitos não funcionais
- Reutilização em larga escala
 - Um componente da arquitetura pode ser reusado em outros sistemas



Arquitetura ruim pode afetar...

- Desempenho
 - Incluir gargalos de comunicação
- Facilidade de distribuição
 - Sistemas que executam em várias máquinas
- Facilidade de manutenção
 - Componentes que devem ser substituíveis

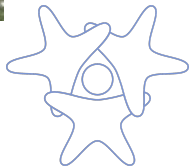
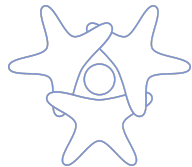
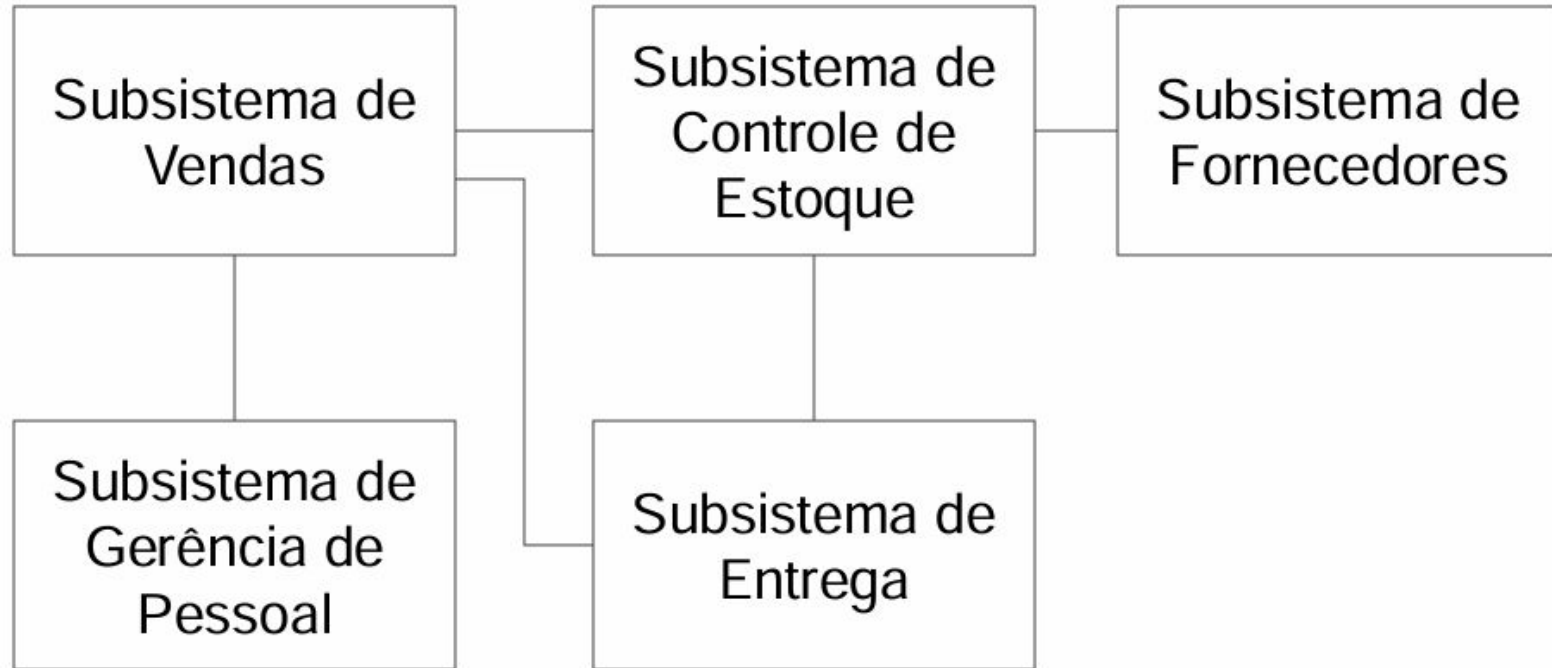


Diagrama de Componentes

- Identifica os principais componentes (subsistemas) a serem desenvolvidos
- Notação simplificada
 - Caixas representam os componentes
 - Linhas representam alguma comunicação entre os componentes
 - Caixas dentro de caixas indicam que um componente foi decomposto



Exemplo de Arquitetura



Quando usar o diagrama?

- Pontos positivos

- Facilita comunicação com stakeholders
- Úteis para planejamento de projeto
- Facilita o desenvolvimento em paralelo

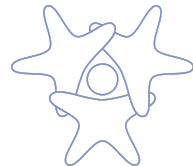
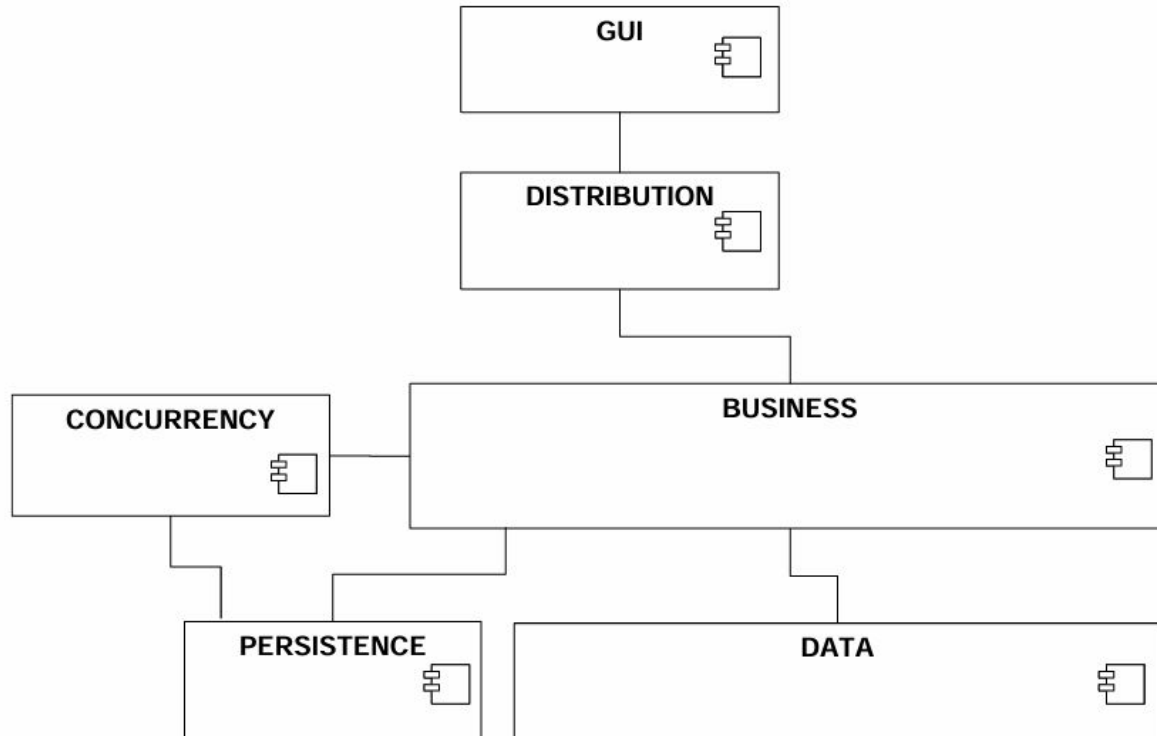


- Pontos negativos

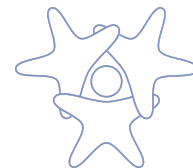
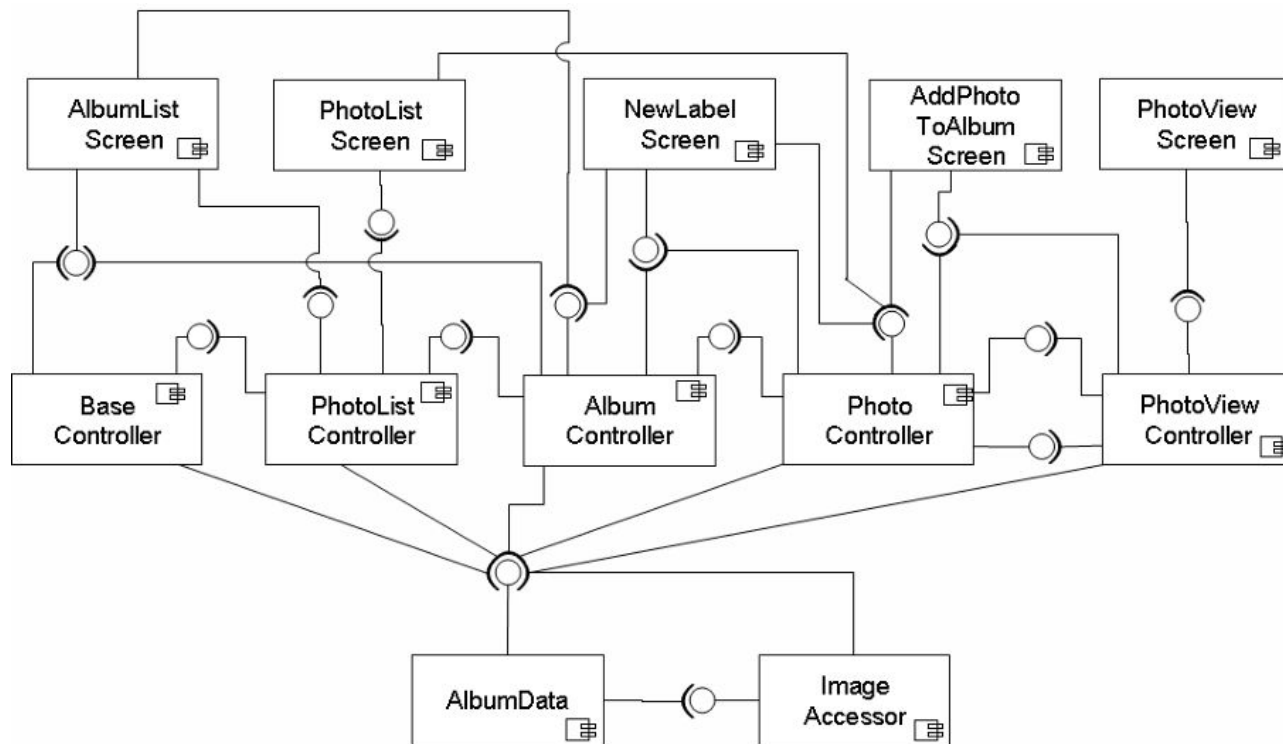
- Não mostra a natureza dos relacionamentos entre componentes
- Não indica as propriedades internas dos subsistemas



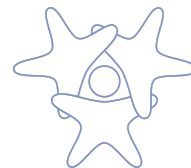
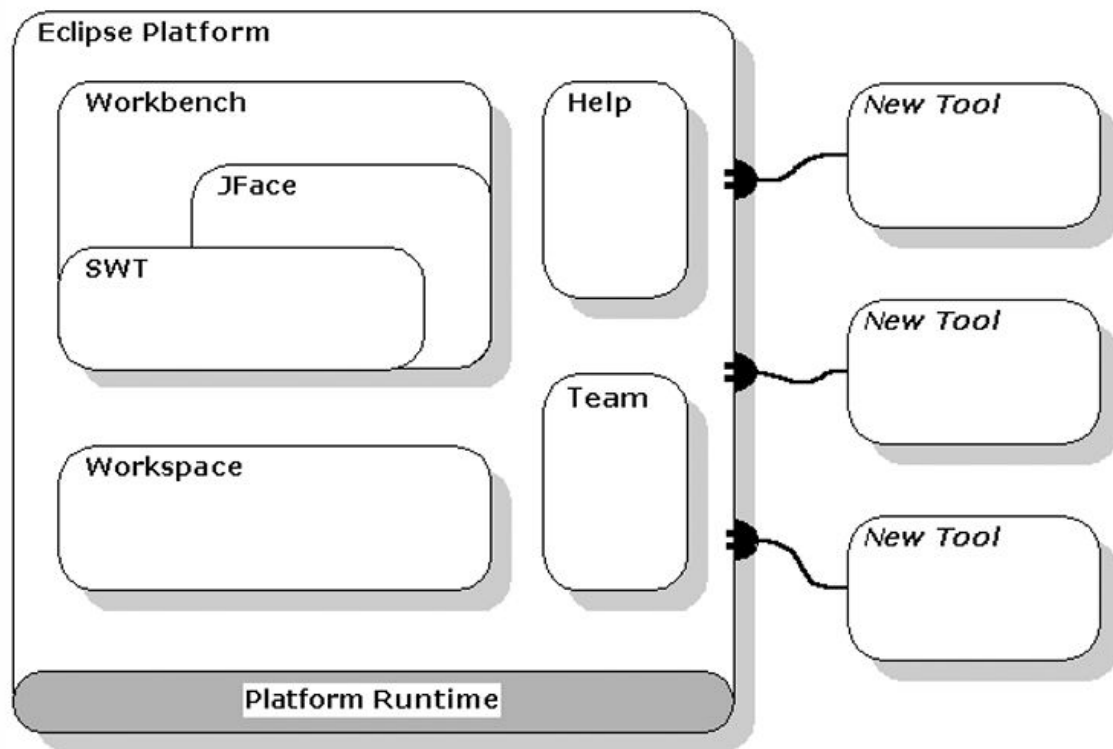
Exemplo de Arquitetura (2)



Exemplo de Arquitetura (3)

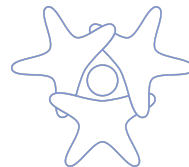


Exemplo de Arquitetura (4)



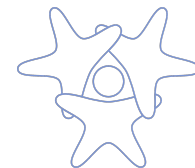
Decisões de projeto arquitetural

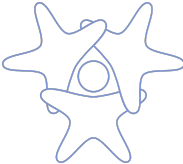
- Questões para serem respondidas
 - Como o sistema será distribuído em diferentes máquinas?
 - Como as funcionalidades serão decompostas em componentes?
 - Qual é a melhor organização para atender aos requisitos não funcionais?
 - Quais padrões ou estilos de arquitetura poderiam ser usados?



Requisitos Não Funcionais

- A arquitetura deve considerar requisitos não funcionais
 - Desempenho: evitar comunicação excessiva entre componentes distribuídos
 - Segurança: ocultar características críticas de segurança em camadas mais internas
 - Disponibilidade: incluir componentes redundantes e tolerância à falhas, etc.



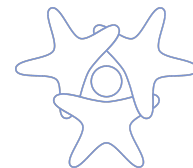


UNINASSAU
Campus Boa Viagem

Padrões de Arquitetura

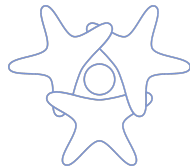
Definição de Padrões

- Um padrão é uma descrição do problema e a essência da sua solução
- Documenta boas soluções para problemas recorrentes
- Deve ser suficientemente abstrato para ser reusado em aplicações diferentes



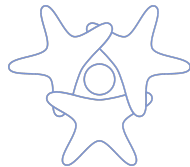
Padrões Arquiteturais

- Padrões arquiteturais expressam formas de organizar a estrutura fundamental do sistema
 - Permitem a construção de uma arquitetura aderente a certas propriedades
 -
- O conhecimento de padrões arquiteturais ajuda na definição da arquitetura do sistema



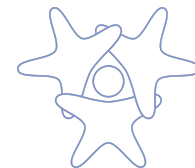
Elementos de um Padrão

- A escolha do padrão arquitetural pode ser o primeiro passo para a solução
- Padrões arquiteturais definem
 - Um conjunto de componentes
 - As responsabilidades de cada componente
 - Regras de relacionamentos entre os componentes



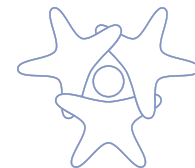
Da arquitetura a implementação

- Os padrões arquiteturais definem a estrutura fundamental
 - Atividades subsequentes devem seguir esta estrutura
- Padrões arquiteturais representam os padrões mais abstratos
 - Padrões de projeto (Modelagem)
 - Idiomas (Programação)



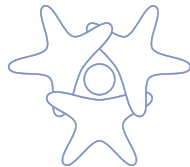
Escolha do Padrão

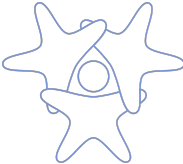
- A escolha do padrão arquitetural deve estar associada ao tipo de sistema e seus requisitos não funcionais
- Algumas perguntas podem ajudar
 - O sistema é interativo?
 - Possui muitas variações?
 - Que requisitos não funcionais são importantes? Confiabilidade? Adaptabilidade?



Composição de Padrões

- Padrões diferentes levam a consequências diferentes
 - Mesmo quando os padrões abordam problemas semelhantes
- Assim como ocorre em padrões de projeto, sistemas complexos possuem mais de um padrão arquitetural



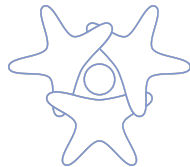


UNINASSAU
Campus Boa Viagem

Padrões Arquiteturais: Da Desordem a Estrutura

Da Desordem a Estrutura

- Layered Architecture
 - Arquitetura em Camadas
- Blackboard
 - Arquitetura de Repositório
- Pipes and Filters
 - Duto e Filtro





UNINASSAU
Campus Boa Viagem

Arquitetura em Camadas

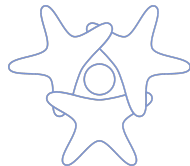
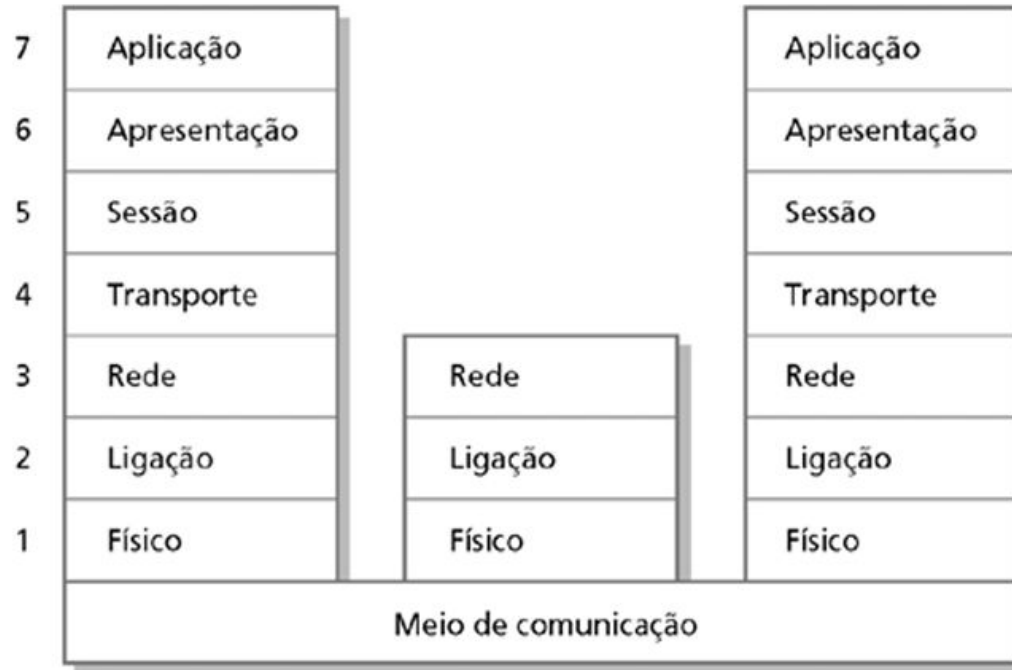
Arquitetura em Camadas

- Organiza o sistema em um conjunto de camadas
 - Cada camada oferece um conjunto de serviços
- Uma camada somente
 - Solicita serviços da camada inferior
 - Fornece serviços para a camada superior

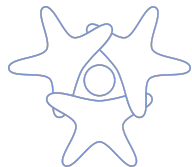
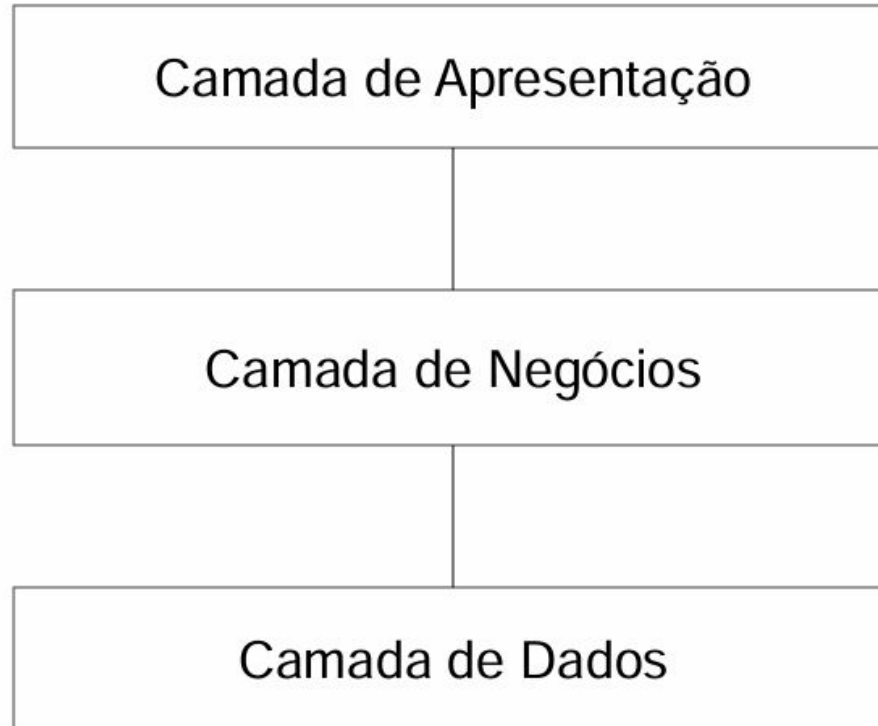


Exemplo 1: Protocolos OSI

- Modelo de camadas para sistemas de comunicação

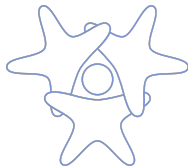


Exemplo 2: Três Camadas



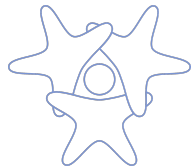
Vantagens

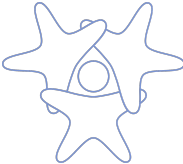
- Favorece o modelo de desenvolvimento incremental
- As camadas podem ser facilmente substituídas por equivalentes
 - Requer interfaces estáveis
- Mudanças em uma camada teoricamente só impacta a camada superior
- Camadas superiores podem ser independentes de plataforma/hardware



Desvantagens

- Estruturar o sistema em camadas não é trivial
 - Pode ser difícil identificar quais os serviços elementares das camadas inferiores
- Muitas camadas podem comprometer o desempenho do sistema
 - A requisição tem que trafegar pelas várias camadas até ser atendida



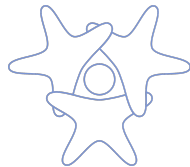
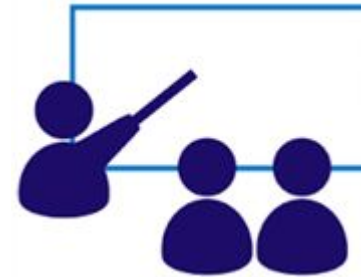


UNINASSAU
Campus Boa Viagem

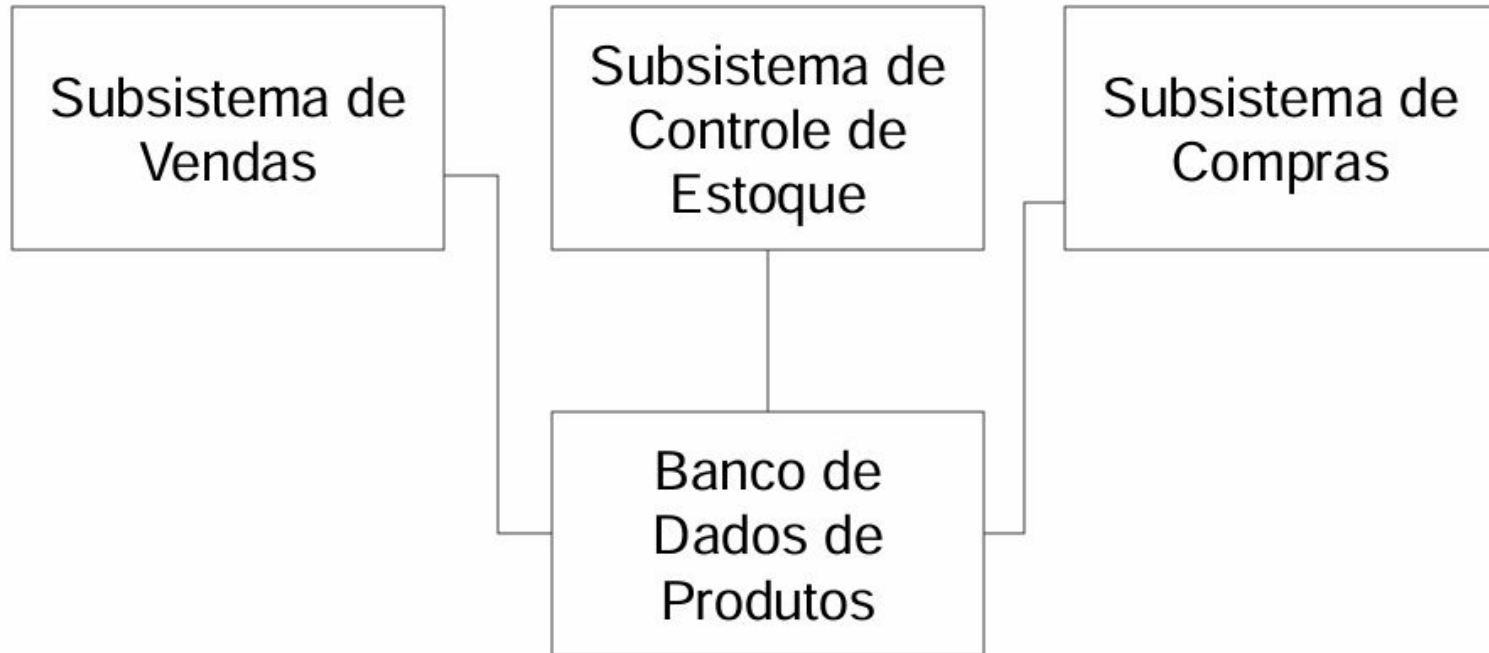
Arquitetura de Repositório

Arquitetura de Repositório

- Também conhecido como Blackboard
- Os subsistemas manipulam a mesma base de dados
 - Um (ou mais) subsistema gera os dados
 - Vários subsistemas leem os dados
- Adotado principalmente quando dados são compartilhados em grandes quantidades

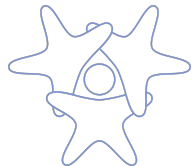


Exemplo de Repositório



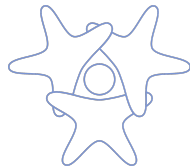
Vantagens

- Componentes podem ser independentes
 - Os subsistemas que gravam dados não necessitam saber quem os usa
- Maneira eficiente de compartilhar dados e backup centralizado
- Formas de proteção dos dados podem ser implementadas
- Fácil integrar novos subsistemas



Desvantagens

- Ponto único de falhas no sistema
 - Problemas no repositório afetam o sistema inteiro
- Ineficiência na organização e comunicação dos componentes
 - Subsistemas diferentes podem ter requisitos diferentes
- Dificuldade para distribuir os dados
 - Dados redundantes ou inconsistentes



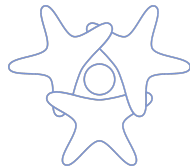


UNINASSAU
Campus Boa Viagem

Duto e Filtro

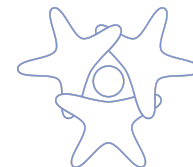
Duto e Filtro

- Padrão de organização da dinâmica de um sistema
- Dois papéis principais
 - Dutos: componentes que conduzem ou distribuem os dados
 - Filtros: componentes que transformam os dados
- Usado principalmente em aplicações de processamento de dados



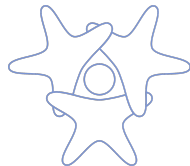
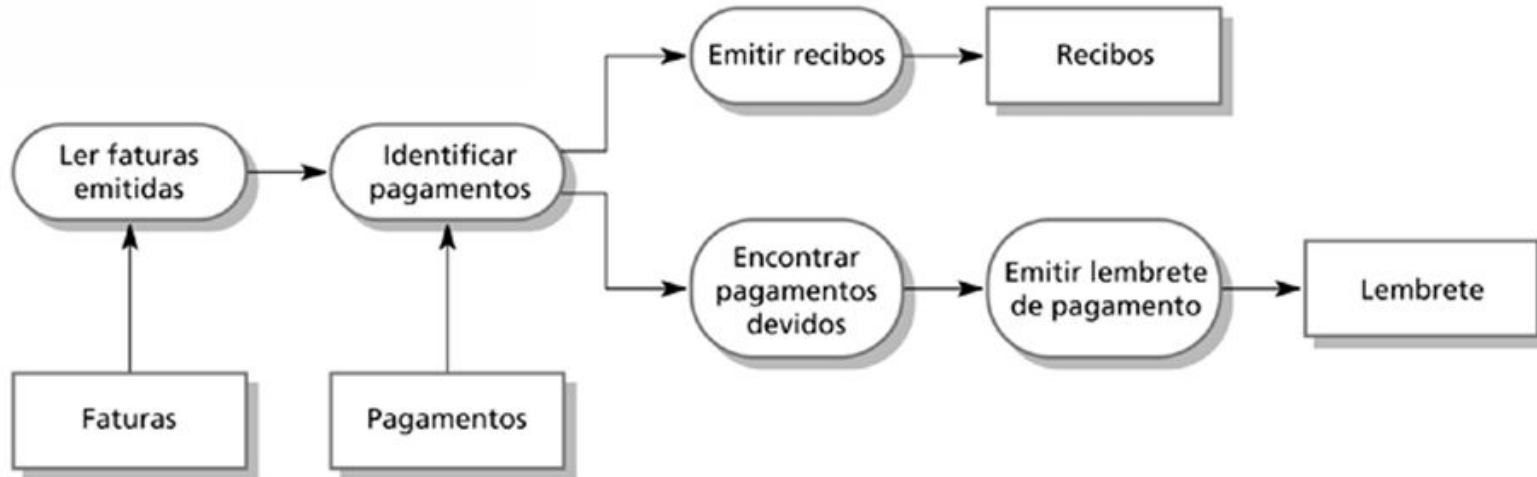
Dinâmica do Padrão

- Os dados de entrada se movem pelos dutos
- Os dados são transformados pelos filtros até serem convertidos em dados de saída
 - As transformações podem ocorrer em sequência ou em paralelo



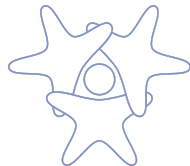
Exemplo de Dutos e Filtros

- Entradas: Faturas e Pagamentos
- Saídas: Recibos e Lembretes



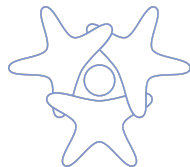
Vantagens

- O módulo de transformação (filtro) é bem modular
 - Facilmente reusável e substituível
- O estilo de workflow é aderente a muitos processos de negócios
- É simples evoluir o sistema pela adição de filtros
- Se aplica tanto a sistemas sequenciais quanto a sistemas concorrentes



Desvantagens

- O formato dos dados trafegados deve ser acordado entre os componentes
- Pode haver um overhead causado pela padronização dos dados
- Incompatibilidade no formato dos dados pode dificultar a reutilização de filtros

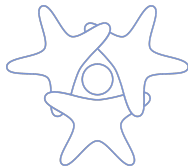


Bibliografia

Eduardo Figueiredo. Engenharia de Software. Departamento de Ciência da Computação da UFMG. www.dcc.ufmg.br/~figueiredo

Ian Sommerville. Engenharia de Software, 10ª Edição. Pearson Education, 2019.

Roger Pressman. Engenharia de Software: Uma Abordagem Profissional, 9a Edição. McGraw-Hill, 2021.





Disciplina: Engenharia de Software e Gestão de Times Ágeis

Fundamentos e Conceitos da
Engenharia de Software

Prof. Pedro Rodolfo Gomes de Souza
prgsouza3@gmail.com

